SELECT *

FORMATION DÉVELOPPEMENT WEB

Bases de données et SQL

Concevoir, créer et interroger une base relationnelle avec MySQL / MariaDB

12 séances · base fil rouge « bibliotheque_lycee » · XAMPP et phpMyAdmin

PROGRAMME

Concevoir, créer, interroger

Partie 1

Concevoir

1 · Donnée, information, base

2 · Le SGBD et phpMyAdmin

3 · Tables et clé primaire

4 · Clés étrangères, associations

5 · Contraintes d'intégrité

6 · Représenter et corriger

Partie 2

Créer et remplir

7 · Créer : CREATE

8 · Modifier, supprimer, exporter

9 · INSERT, UPDATE, DELETE

Partie 3

Interroger

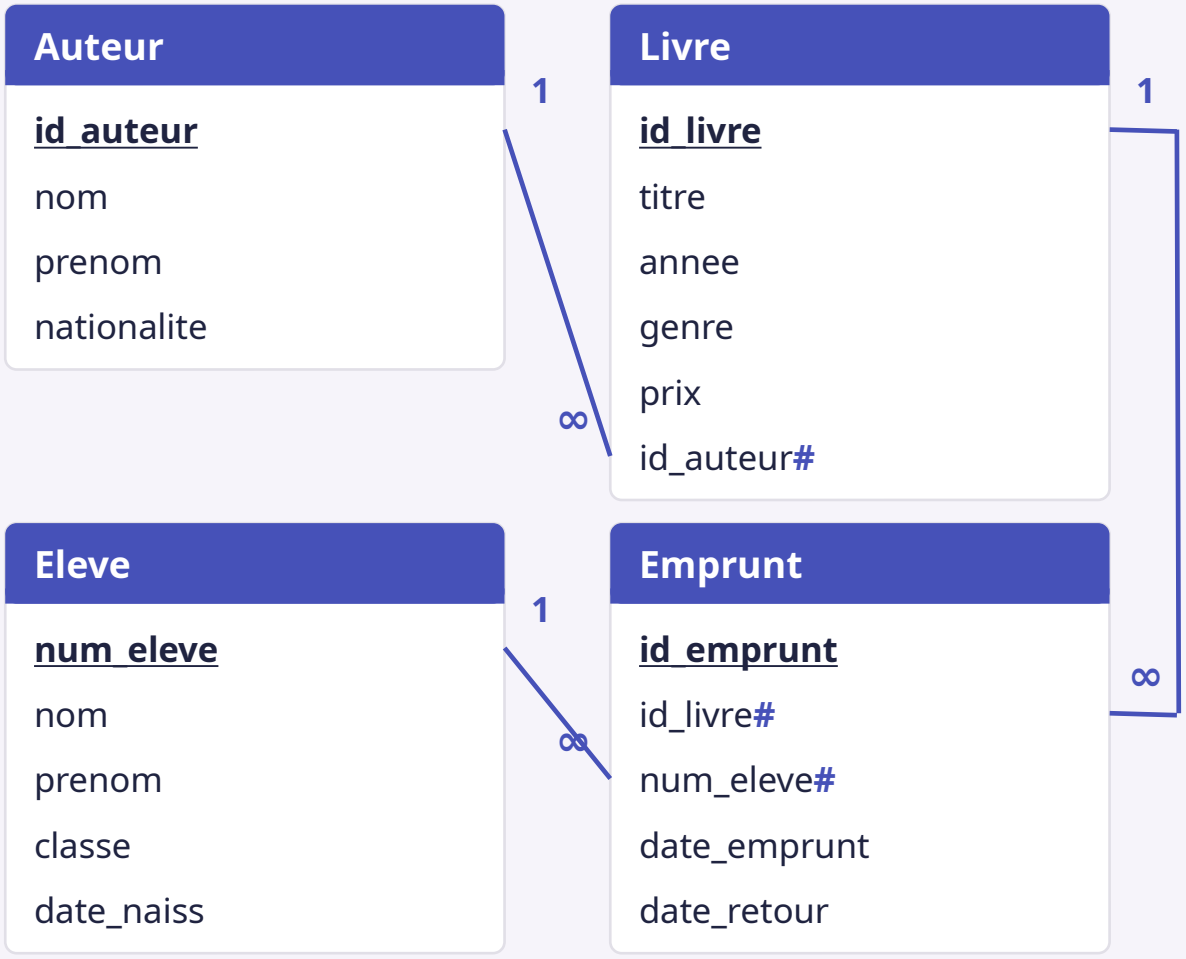
10 · SELECT et opérateurs

11 · Dates, agrégats, GROUP BY

12 · Jointures, sous-requêtes

BASE FIL ROUGE

La bibliothèque du lycée



Mettre la base en place

- phpMyAdmin → Importer → bibliotheque_lycee.sql
- Le script supprime puis recrée la base : on peut le relancer
- Clé primaire soulignée, clé étrangère suivie de #

6	8	6	8
auteurs	livres	élèves	emprunts

SÉANCE 1

Donnée, information et base de données

Chapitre 1, section 1.1

Objectifs

- Distinguer donnée et information
- Décrire une entité par ses attributs
- Définir une base de données et son utilité

Une donnée seule n'a pas de sens

DONNÉE

14

Quatorze quoi ? Pour qui ?



INFORMATION

Leila Ben Salah a obtenu 14 en Informatique.

élève · note · matière : le contexte donne le sens

Un ensemble de données lié à un contexte donne une information.

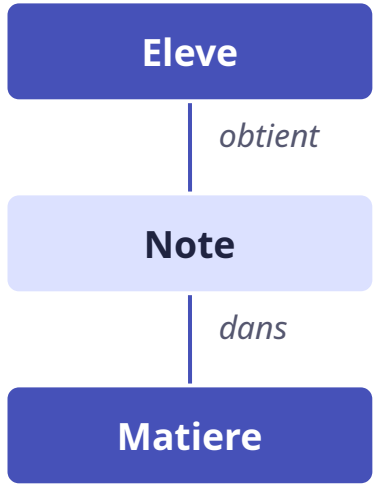
Entité, attribut, valeur, domaine

Attribut	Valeur	Domaine
nom	Ben Salah	Texte, 50 caractères au plus
prenom	Leila	Texte, 50 caractères au plus
classe	4SI1	Les classes du lycée
date_naiss	2008-03-14	Une date

Entité : Eleve

- Entité : objet du monde réel (personne, fait, objet)
- Attribut : propriété de l'entité ; valeur : son contenu
- Domaine : ensemble des valeurs possibles (note : réel de 0 à 20)

Une association relie des entités



Structurée, non redondante, persistante

1

Structurée

Rangée en tables, colonnes et lignes

Le nom de l'élève est toujours dans la colonne nom

2

Non redondante

Chaque donnée n'est saisie qu'une fois

Le nom d'un auteur n'est pas recopié dans chaque livre

3

Persistante

Conservée durablement, non volatile

Les emprunts restent après l'arrêt du serveur

Exemple : la base « bibliotheque_lycee » — auteurs, livres, élèves, emprunts.

Pourquoi une base plutôt que des fichiers

Centralisation

Une seule copie des données, utilisée par tous les programmes

Partage

Administration, professeurs et parents consultent les mêmes notes

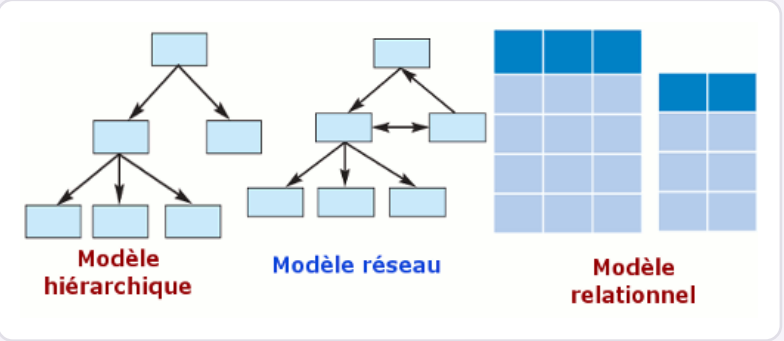
Cohérence

Une donnée modifiée une fois est à jour partout

Indépendance

Les programmes changent sans réorganiser les données

Modèles de bases



Dans ce cours : le modèle relationnel

À VOUS DE JOUER

Exercices

Ex. 1

Location de voitures

Lister les données nécessaires pour chaque véhicule d'une agence.

Ex. 2

Donnée ou information ?

Classer : « 25.500 », « Les Misérables coûte 25.500 DT », « 4SI1 », « Leila est en 4SI1 ».

Ex. 3

Magasin de sport

Identifier deux entités et trois attributs pour chacune, avec leur domaine.

À retenir

- Donnée + contexte = information
- Entité décrite par des attributs ; valeur dans un domaine
- Base : structurée, non redondante, persistante
- Modèle étudié : relationnel

SÉANCE 2

Le SGBD, XAMPP et phpMyAdmin

Chapitre 1, section 1.2 · Annexe A.3

Objectifs

- Définir un SGBD et ses deux parties
- Citer les fonctions d'un SGBD
- Importer la base fil rouge dans phpMyAdmin

Le SGBD : une interface et un moteur

Utilisateur

écrit une requête

→

Interface

phpMyAdmin

→

Moteur

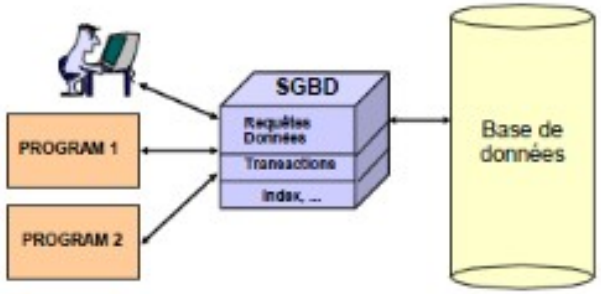
MariaDB / MySQL

→

Base

bibliotheque_lycee

SGBD : logiciel qui permet de créer, modifier, interroger et administrer une base de données. Le moteur assure les fonctions ; l'interface rend l'accès facile et convivial.



Six fonctions d'un SGBD

Fonction	Rôle	Exemple au lycée
Définition	Créer les tables, leurs colonnes et leurs contraintes	Créer la table Eleve
Manipulation	Ajouter, modifier, supprimer, rechercher des données	Saisir les notes du trimestre
Intégrité	Refuser les données qui violent les contraintes	Refuser une note de 25 sur 20
Accès concurrents	Plusieurs utilisateurs travaillent en même temps	Deux professeurs saisissent leurs notes
Confidentialité	Contrôler qui peut voir ou modifier	Un élève voit ses notes sans les modifier
Sécurité	Sauvegarder et restaurer après une panne	Restaurer après une coupure de courant

Des SGBD relationnels partout

MySQL | **MariaDB** | **PostgreSQL** | **Oracle**

SQL Server | **DB2** | **Access**

En bleu : les SGBD utilisés avec XAMPP dans ce cours

Hors programme

MongoDB n'est pas relationnel : base NoSQL, sans tables.

Relationnel = données rangées dans des tables reliées entre elles.

ANNEXE A.3 · PRATIQUE

Importer la base dans phpMyAdmin

1. XAMPP : démarrer Apache et MySQL
2. Ouvrir localhost/phpmyadmin
3. Onglet Importer : bibliotheque_lycee.sql
4. Cliquer sur Importer, puis vérifier

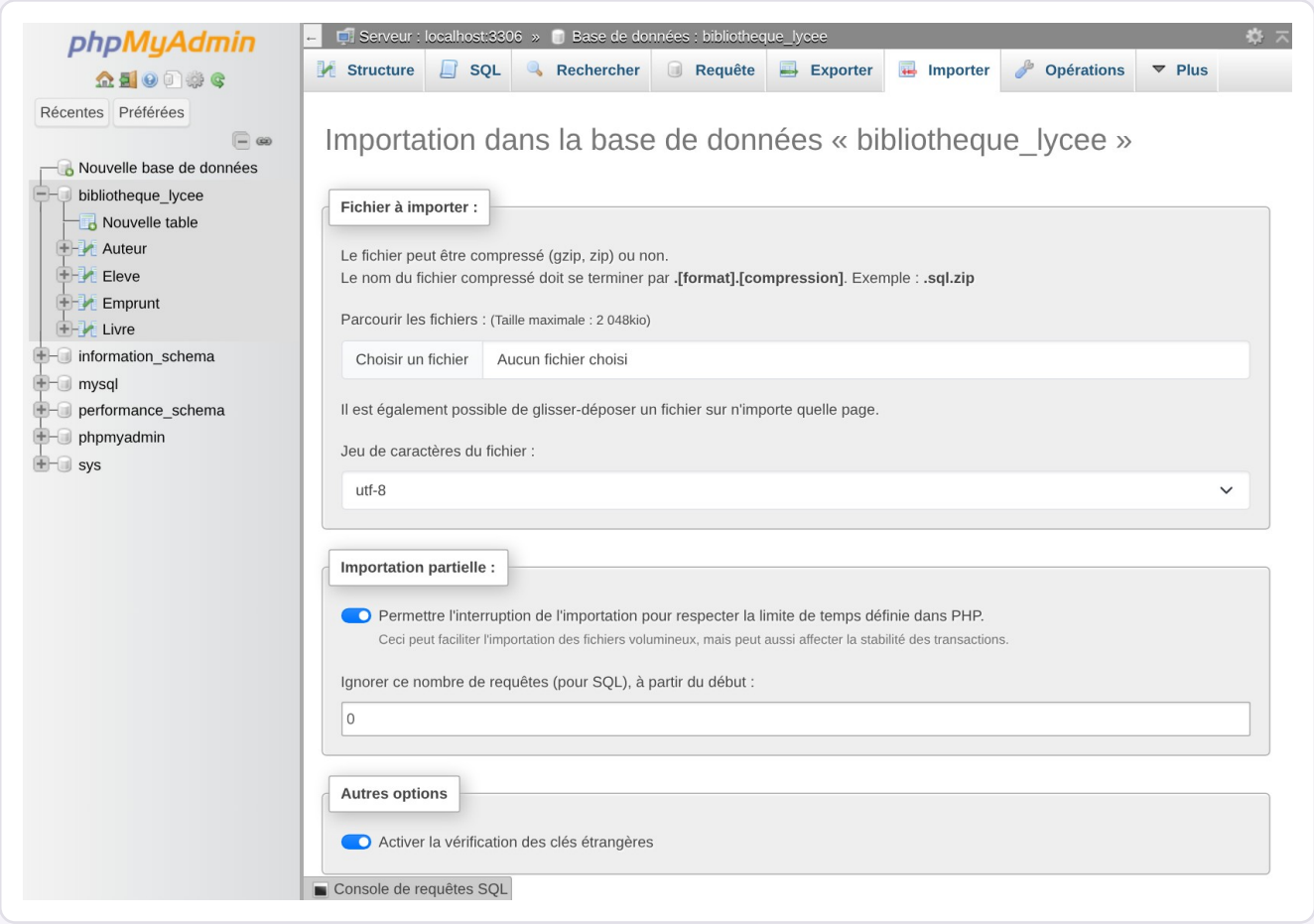
```sql
-- Requête de contrôle (onglet SQL)
SELECT
  (SELECT COUNT(*) FROM Auteur)  AS nb_auteurs,
  (SELECT COUNT(*) FROM Livre)   AS nb_livres,
  (SELECT COUNT(*) FROM Eleve)   AS nb_eleves,
  (SELECT COUNT(*) FROM Emprunt) AS nb_emprunts;
```

nb_auteurs	nb_livres	nb_eleves	nb_emprunts
6	8	6	8

Résultat attendu

ANNEXE A.3 · PRATIQUE

L'onglet Importer de phpMyAdmin



- 1 · Sélectionner la base (ou rien si le fichier contient CREATE DATABASE)
- 2 · Choisir un fichier : bibliotheque_lycee.sql
- 3 · Importer, en bas de page
- 4 · Lire le message : nombre de requêtes exécutées

À VOUS DE JOUER

Exercices

Ex. 1

Quelle fonction ?

Associer à une fonction du SGBD : refuser une note de 25, restaurer après une panne, deux saisies simultanées.

Ex. 2

Moteur ou interface ?

Classer : phpMyAdmin, MariaDB, la fenêtre SQL, le stockage des tables.

Ex. 3

Mise en place

Importer bibliotheque_lycee.sql et obtenir 6 / 8 / 6 / 8 avec la requête de contrôle.

À retenir

- SGBD = moteur + interface
- Six fonctions : définition, manipulation, intégrité, accès concurrents, confidentialité, sécurité
- XAMPP : phpMyAdmin (interface) + MariaDB (moteur)

SÉANCE 3

Tables, colonnes et clé primaire

Chapitre 1, section 1.3

Objectifs

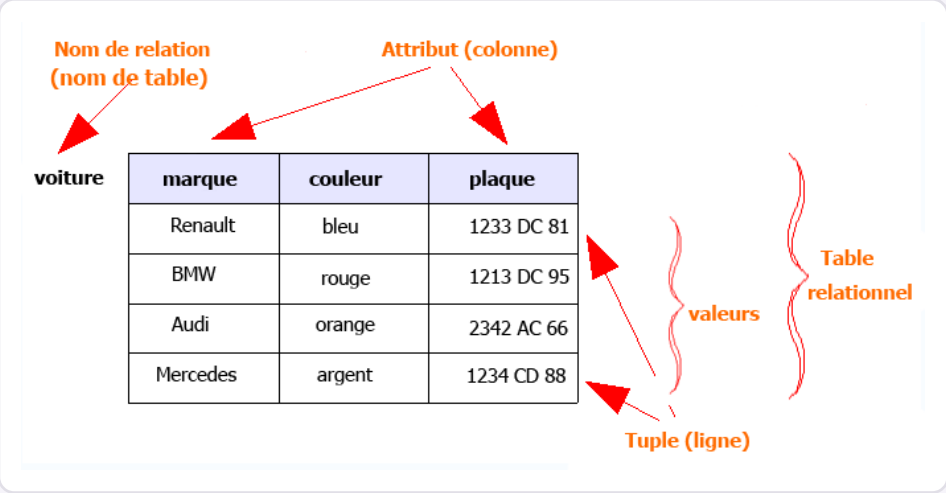
- Lire une table : colonnes et lignes
- Décrire une colonne : type, taille, contraintes
- Choisir une clé primaire

Une table range une entité en tableau

Table Article

code_art	designation	prix_unitaire	qte_stock
V10	Vis 50*3	40	2500
V20	Vis 20*2	20	1300
B100	Boulon 90*15	450	100
C60	Clou 60*2	50	5000

- Colonne = attribut, champ : une propriété (prix_unitaire)
- Ligne = enregistrement, n-uplet : une occurrence (la vis V10)



Table, attributs, tuples et valeurs

Les mots de la base relationnelle

Terme	Synonymes	Exemple (base bibliothèque)
Table	Relation	Livre
Colonne	Champ, attribut	titre
Ligne	Enregistrement, n-uplet, tuple	(3, 'L''Etranger', 1942, ...)
Clé primaire	Identifiant	id_livre
Clé étrangère	Référence vers une autre table	id_auteur# dans Livre

Une colonne se décrit en six points

Élément	Mot-clé SQL	Exemple
Un nom	—	nom
Un type	INT, VARCHAR, DATE…	prix DECIMAL(6,3)
Une taille	(n)	nom VARCHAR(50)
Obligatoire ?	NOT NULL	classe … NOT NULL
Valeur par défaut	DEFAULT	DEFAULT 'Inconnue'
Une règle	CHECK	CHECK (prix > 0)

NULL

Absence de valeur.

NULL ≠ 0

NULL ≠ ''

La date de retour d'un livre non rendu vaut NULL.

Décrire toutes les colonnes d'une table

Nom de la table : Article
Description : Détail des articles commercialisés

Nom colonne	Description	Type de données	Taille	Obligatoire	Valeur par défaut	Valeurs autorisées
Code_art	Code de l'article	Chaîne de caractères	20	Oui		
Des_art	Désignation de l'article	Chaîne de caractères	50	Oui		
PU	Prix unitaire de l'article	Numérique	8,3	Non		> 0
Qte_stok	Quantité en stock	Numérique	4	Non	0	>= 0

Pour chaque colonne

- Type et taille
- Obligatoire → NOT NULL
- Valeur par défaut → DEFAULT
- Valeurs autorisées → CHECK

La clé primaire identifie chaque ligne

Livre
id_livre
titre
annee
genre
prix
id_auteur#

- Unique : deux lignes n'ont jamais la même valeur
- Toujours renseignée : jamais NULL
- Une seule par table, d'une ou plusieurs colonnes
- **Convention : clé primaire soulignée**

Bonne ou mauvaise clé ?

num_eleve	**Bonne**
nom	**Doublons**
code_art	**Bonne**
classe	**Doublons**

Quelle colonne identifie l'entité ?

Activité

1) Dans chacun des tableaux suivants, cocher chaque case permettant d'identifier d'une manière unique l'entité :

Citoyen	Nom	Prénom	Nom et Prénom	Age	N° CIN	N° Sécurité Sociale	Adresse

Voiture	Marque	Modèle	Marque et modèle	Age	N° Immatriculation	N° châssis	Lieu de fabrication

Citoyen et voiture

Etudiant	Nom	Prénom	Nom et Prénom	Age	N° CIN	N° Carte Etudiant	Adresse

Elève	Nom	Prénom	Nom et Prénom	Age	N° Téléphone	N° Inscription	Adresse

Ticket de transport	Prix	Ville de départ	Ville d'arrivée	Date	N° Ticket	Société de transport	Validité

Étudiant, élève, ticket de transport

À VOUS DE JOUER

Exercices

Ex. 1

Lire la table Livre

Nommer ses colonnes, compter ses lignes dans phpMyAdmin, donner sa clé primaire.

Ex. 2

Choisir une clé

Proposer la clé primaire de Eleve, Auteur et Article ; justifier chaque choix.

Ex. 3

Décrire les colonnes

Pour chaque colonne de Eleve : type, taille, obligatoire ou non.

À retenir

- Table = colonnes (attributs) × lignes (enregistrements)
- Colonne : nom, type, taille, NOT NULL, DEFAULT, CHECK
- Clé primaire : unique, non nulle, soulignée
- NULL = absence de valeur

SÉANCE 4

Clés étrangères et associations

Chapitre 1, section 1.3

Objectifs

- Relier deux tables par une clé étrangère
- Distinguer table mère et table fille
- Traduire les associations 1-1, 1-N et N-N

La clé étrangère relie deux tables

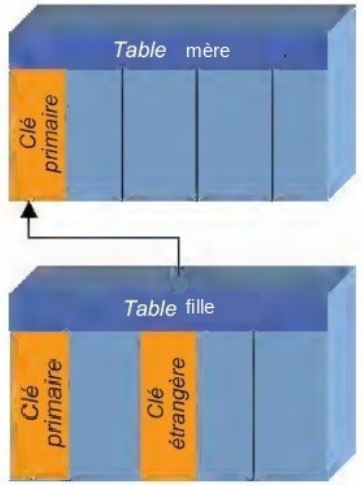
Etablissement
CodeEtab
NomEtab
AdresseEtab
TelEtab

Table mère

1 — ∞

Eleve
NumElev
NomElev
PrenomElev
DnaissElev
CodeEtab#

Table fille

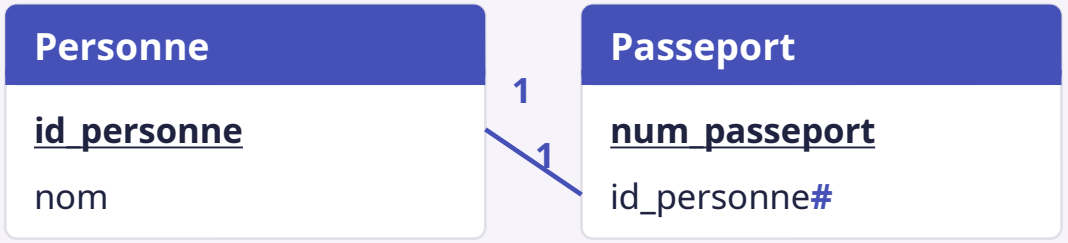


CodeEtab# référence CodeEtab

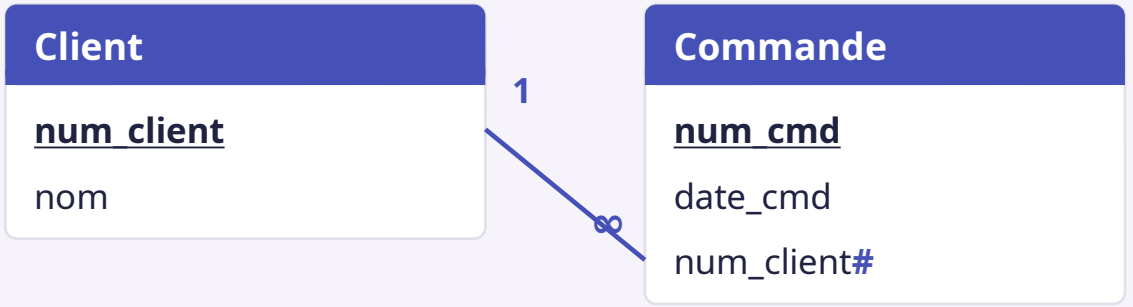
1 côté mère, ∞ côté fille

1 à 1 et 1 à N : où placer la clé étrangère

1 à 1

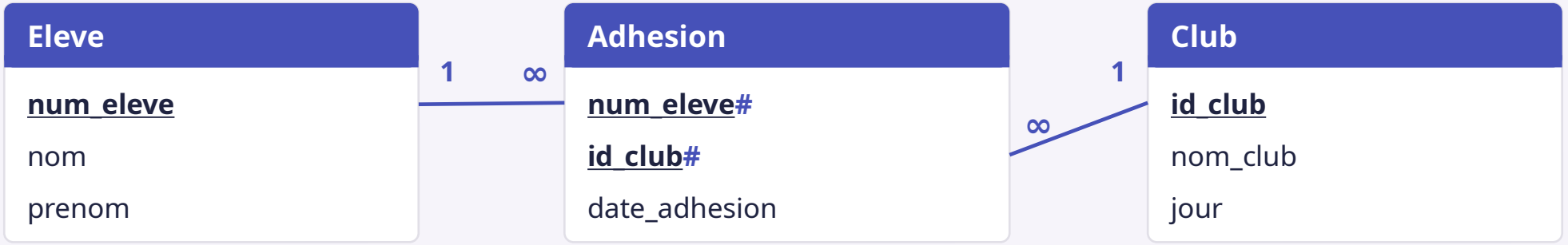


1 à N



Règle 1 à N : la clé primaire du côté 1 devient clé étrangère dans la table du côté N. Un client, plusieurs commandes : num_client# va dans Commande.

N à N : une table intermédiaire



Eleve (num_eleve, nom, prenom)
Club (id_club, nom_club, jour)
Adhesion (num_eleve#, id_club#, date_adhesion)

Clé primaire composée

num_eleve# et id_club# sont soulignés tous les deux.

Lire les associations de la bibliothèque

Association	Type	Traduction
Un auteur écrit des livres	**1 à N**	id_auteur# dans Livre
Un élève emprunte des livres	**N à N**	Emprunt (id_emprunt, id_livre#, num_eleve#, …)
Un emprunt concerne un livre	**N à 1**	id_livre# dans Emprunt
Un emprunt concerne un élève	**N à 1**	num_eleve# dans Emprunt

Mères : Auteur, Eleve · Filles : Livre (de Auteur), Emprunt (de Livre et de Eleve)

À VOUS DE JOUER

Exercices

Ex. 1

Clients et commandes

Placer la clé étrangère entre Client et Commande ; justifier par le type d'association.

Ex. 2

Élèves et clubs

Représenter l'association N à N avec sa table intermédiaire.

Ex. 3

Mère ou fille ?

Pour chaque table de la bibliothèque, dire si elle est mère, fille ou les deux.

À retenir

- Clé étrangère suivie de #
- Elle référence la clé primaire de la table mère
- 1 à N : la clé étrangère va côté N
- N à N : table intermédiaire

SÉANCE 5

Les contraintes d'intégrité

Chapitre 2, section 2.1

Objectifs

- Citer les trois types de contraintes
- Associer chaque contrainte à ses mots-clés SQL
- Expliquer pourquoi le SGBD refuse une donnée

Trois contraintes, trois gardiens

Domaine

La valeur respecte le type et la condition de sa colonne

Exemple

âge entier et âge ≥ 18 : la valeur 15 est refusée

Table

Clé primaire unique et renseignée ; UNIQUE sans doublon

Exemple

deux élèves avec le numéro 101 : refusé

Référentielle

La clé étrangère existe comme clé primaire de la table mère

Exemple

emprunt de l'élève 999 qui n'existe pas : refusé

Une contrainte empêche la base de stocker des données incohérentes.

Ce que chaque contrainte refuse

Domaine

```
age INT CHECK (age >= 18)
```

Etudiant

id_etudiant	nom	age
1	Ben Ali	20
2	Trabelsi	19
3	Kefi	15

Insertion refusée
age = 15 ne respecte pas age >= 18

Référentielle

```
Inscription : FOREIGN KEY (id_etudiant) REFERENCES Etudiant(id_etudiant)
```

Etudiant

id_etudiant	nom
1	Ben Ali
2	Trabelsi
3	Kefi

Inscription

id_etudiant#	code_cours
1	BD01
2	RES02
3	BD01
999	BD01

Insertion refusée
999 n'existe pas dans Etudiant

Table

```
id_etudiant INT PRIMARY KEY,  email VARCHAR(100) UNIQUE
```

Etudiant

id_etudiant	nom	email
1	Ben Ali	ali@ecole.tn
2	Trabelsi	karim@ecole.tn
2	Kefi	sami@ecole.tn
4	Gharbi	ali@ecole.tn

Refusée : id 2 existe déjà (clé primaire)
Refusée : email déjà utilisé (UNIQUE)

Des contraintes aux mots-clés SQL

Contrainte	Mots-clés SQL	Exemple
Domaine	type, NOT NULL, DEFAULT, CHECK	note DECIMAL(4,2) CHECK (note BETWEEN 0 AND 20)
Table	PRIMARY KEY, UNIQUE	email VARCHAR(100) UNIQUE
Référentielle	FOREIGN KEY … REFERENCES	FOREIGN KEY (id_auteur) REFERENCES Auteur(id_auteur)

ON DELETE CASCADE / ON UPDATE CASCADE complètent la contrainte référentielle : la suppression ou la modification de la ligne mère se répercute sur les lignes filles.

Le SGBD refuse toute donnée incohérente

Tentative	Contrainte violée
Ajouter un livre avec id_livre = 1 (déjà utilisé)	**Table : clé primaire**
Ajouter un livre de l'auteur n° 99, inexistant	**Référentielle**
Ajouter un livre au prix de -5	**Domaine : CHECK (prix > 0)**
Supprimer un auteur qui a écrit des livres	**Référentielle (table mère)**

```
INSERT INTO Livre VALUES
 (9, 'Notre-Dame de Paris',
  1831, 'Roman', 15, 99);
```

Refusé : l'auteur n° 99 n'existe pas

À VOUS DE JOUER

Exercices

Ex. 1

Quelle contrainte ?

Nommer la contrainte violée : note de 25, deux élèves n° 101, emprunt du livre 99, email en double.

Ex. 2

La table Note

Proposer les contraintes de Note (num_eleve#, code_mat#, trimestre, note).

Ex. 3

Supprimer un auteur

Expliquer pourquoi la suppression de Victor Hugo est refusée.

À retenir

- Domaine : type et condition (NOT NULL, DEFAULT, CHECK)
- Table : PRIMARY KEY, UNIQUE
- Référentielle : la clé étrangère référence une clé primaire existante

SÉANCE 6

Représenter et corriger une base

Chapitre 2, sections 2.2 et 2.3

Objectifs

- Écrire la représentation textuelle d'une base
- Passer du graphique au textuel et inversement
- Évaluer et corriger une représentation

Une même base, deux écritures

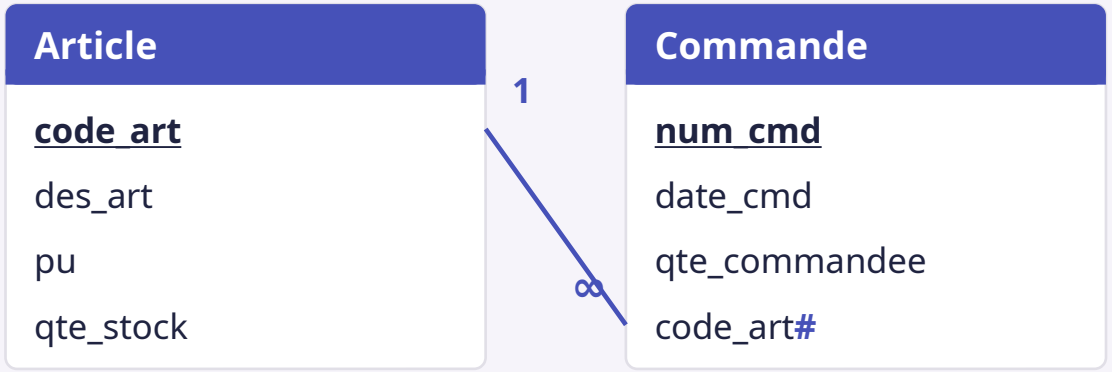
Textuelle

Article (<u>code_art</u>, des_art, pu, qte_stock)

Commande (<u>num_cmd</u>, date_cmd, qte_commandee, code_art#)

- Soulignée : clé primaire
- Suivie de # : clé étrangère
- Lien : 1 côté table mère, ∞ côté table fille

Graphique



Convertir en quatre étapes

Graphique → textuelle

1. Une ligne Nom_table (...) par rectangle
2. Les colonnes dans l'ordre, la clé primaire soulignée
3. Colonne reliée côté ∞ : ajouter #
4. Vérifier : chaque # correspond à un lien

Textuelle → graphique

1. Un rectangle par table, titré par son nom
2. Une ligne par colonne, la clé primaire soulignée
3. Pour chaque X#, un lien vers la clé primaire référencée
4. Écrire 1 côté mère, ∞ côté fille

Convertir : la réservation d'hôtel



Client (num_client, nom, prenom, telephone)

Chambre (num_chambre, type, prix_nuit, etage)

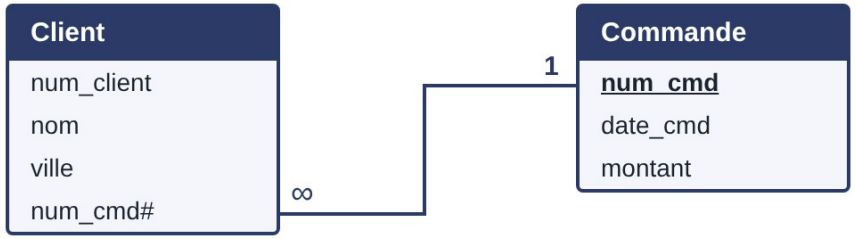
Reservation (num_resa, date_arrivee, nb_nuits, num_client#, num_chambre#)

Six vérifications avant de valider

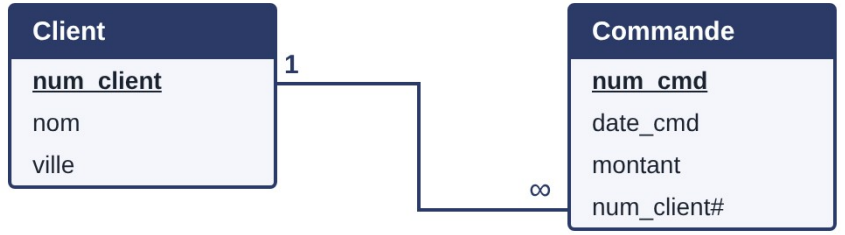
N°	Vérification	Anomalie typique
1	Chaque table a une clé primaire, soulignée	Aucune colonne soulignée
2	La clé primaire identifie vraiment chaque ligne	Clé primaire = nom (homonymes possibles)
3	Chaque clé étrangère suivie de # et liée à une clé primaire	Lien vers une colonne qui n'est pas clé primaire
4	1 à N : la clé étrangère est du côté N	num_cmd# placé dans Client
5	N à N : une table intermédiaire	Clés étrangères croisées entre les deux tables
6	Ni redondance ni colonne contenant une liste	Nom du client dans chaque commande ; liste_produits

Clients et commandes : la clé du mauvais côté

Erronée



Corrigée



- Aucune clé primaire dans Client : souligner num_client
- 1 à N : la clé étrangère va dans Commande (côté N), pas dans Client

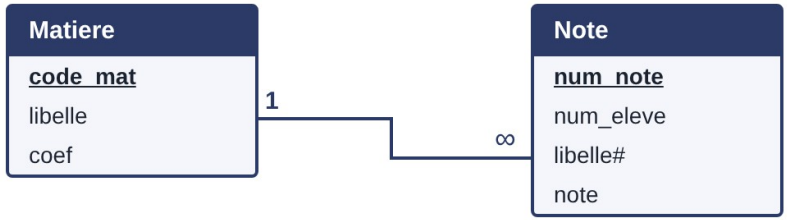
Deux autres erreurs fréquentes

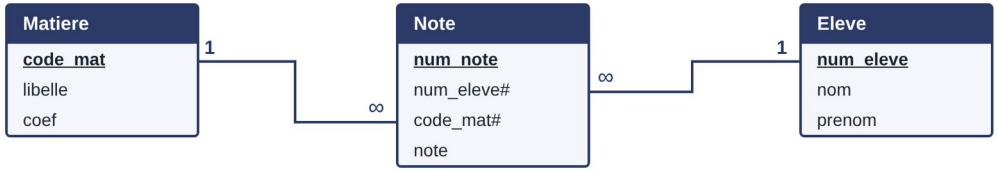
Cas 2 · Élèves et clubs



- Association N à N traduite par des clés croisées
- **Correction : table intermédiaire Adhesion**

Cas 3 · Matières et notes





Corrigée

Corriger une représentation erronée

Erronée

Commande (num_cmd, date_cmd, nom_client, tel_client, liste_produits**)**

- Aucune clé primaire soulignée
- Redondance : client recopié dans chaque commande
- liste_produits contient plusieurs valeurs : association N à N

Corrigée

Client (num_client, nom_client, tel_client**)**

Produit (code_prod, designation, prix**)**

Commande (num_cmd, date_cmd, num_client**#)**

Ligne_commande (num_cmd#, **code_prod#**, quantite**)**

À VOUS DE JOUER

Exercices

Ex. 1

Réservation d'hôtel

Écrire la représentation textuelle du schéma Client, Chambre, Reservation.

Ex. 2

La bibliothèque

Dessiner la représentation graphique des quatre tables à partir du texte.

Ex. 3

Clients et commandes

Repérer deux anomalies dans le schéma fourni, puis le corriger.

À retenir

- Textuelle : Nom_table (colonnes), clé soulignée, #
- Graphique : rectangles, liens 1 → ∞
- Vérifier avec la liste en six points
- Pas de redondance, pas de liste dans une colonne

SÉANCE 7

Créer une base et ses tables

Chapitre 3, sections 3.1 à 3.3

Objectifs

- Situer les quatre sous-langages de SQL
- Choisir le type d'une colonne
- Créer une base et ses tables avec leurs contraintes

SQL : quatre sous-langages

LDD

Définition

Structure des tables

CREATE · ALTER · DROP

Séances 7-8

LMD

Manipulation

Contenu des tables

INSERT · UPDATE · DELETE

Séance 9

LID

Interrogation

Lire les données

SELECT

Séances 10-12

LCD

Contrôle

Droits d'accès

GRANT · REVOKE

Non étudié

Choisir le type de chaque colonne

Type	Signification	Exemple
INT	Entier	num_eleve INT
DECIMAL(p,d)	Réel : p chiffres dont d décimales	25.500
CHAR(n)	Chaîne de longueur fixe	code_postal CHAR(4)
VARCHAR(n)	Chaîne variable, n caractères au plus	nom VARCHAR(50)
TEXT	Texte long	resume TEXT
DATE	Date AAAA-MM-JJ	'2026-10-13'
TIME	Heure HH:MM:SS	'08:30:00'
DATETIME	Date et heure	'2026-10-13 08:30:00'

Avec XAMPP

- Année : INT + CHECK (YEAR refuse 1862)
- Liste de valeurs : VARCHAR + CHECK … IN plutôt qu'ENUM
- AUTO_INCREMENT : hors standard

CREATE DATABASE, puis CREATE TABLE

```sql
CREATE DATABASE bibliotheque_lycee;
USE bibliotheque_lycee;

CREATE TABLE Auteur (
  id_auteur   INT PRIMARY KEY,
  nom         VARCHAR(50) NOT NULL,
  prenom      VARCHAR(50),
  nationalite VARCHAR(30) DEFAULT 'Inconnue'
);
```

Contrainte	Rôle
NOT NULL	Valeur obligatoire
DEFAULT	Valeur par défaut
CHECK	Condition à respecter
PRIMARY KEY	Clé primaire
UNIQUE	Pas de doublon
FOREIGN KEY … REFERENCES	Clé étrangère
ON DELETE CASCADE	Suppression en cascade

Les tables mères avant les tables filles

1 · Auteur

2 · Eleve

3 · Livre

4 · Emprunt

mères, puis filles

```
CREATE TABLE Emprunt (
  id_emprunt   INT PRIMARY KEY,
  id_livre     INT NOT NULL,
  num_eleve    INT NOT NULL,
  date_emprunt DATE NOT NULL,
  date_retour  DATE,
  CONSTRAINT fk_emprunt_livre FOREIGN KEY (id_livre) REFERENCES Livre(id_livre),
  CONSTRAINT fk_emprunt_eleve FOREIGN KEY (num_eleve) REFERENCES Eleve(num_eleve)
    ON DELETE CASCADE,
  CONSTRAINT chk_dates CHECK (date_retour >= date_emprunt)
);
```

À VOUS DE JOUER

Exercices

Ex. 1

Auteur et Eleve

Écrire CREATE TABLE pour les deux tables mères, avec leurs contraintes.

Ex. 2

La table Livre

Écrire CREATE TABLE Livre : année entre 1500 et 2100, prix positif, clé étrangère nommée.

Ex. 3

Pourquoi CASCADE ?

Expliquer l'effet de ON DELETE CASCADE sur Emprunt quand on supprime un élève.

À retenir

- LDD : CREATE, ALTER, DROP
- Choisir type et taille de chaque colonne
- Contraintes dans CREATE TABLE
- Tables mères d'abord, tables filles ensuite

SÉANCE 8

Modifier, supprimer et sauvegarder

Chapitre 3, sections 3.4 et 3.5 · Annexe

Objectifs

- Modifier la structure d'une table avec ALTER
- Supprimer tables et bases dans le bon ordre
- Exporter et importer une base au format SQL

ALTER TABLE change la structure

```
-- Ajouter une colonne
ALTER TABLE Eleve ADD telephone VARCHAR(8);
-- Supprimer une colonne
ALTER TABLE Eleve DROP COLUMN date_naiss;
-- Modifier le type (répéter NOT NULL)
ALTER TABLE Eleve MODIFY nom VARCHAR(80) NOT NULL;
-- Renommer
ALTER TABLE Eleve RENAME COLUMN classe TO niveau;
ALTER TABLE Eleve CHANGE niveau classe VARCHAR(10) NOT NULL;
```

Attention

MariaDB et MySQL (XAMPP) refusent :

ALTER col type

RENAME nom1 TO nom2

Utiliser MODIFY, RENAME COLUMN ou CHANGE.

Ajouter, supprimer, suspendre une contrainte

```
ALTER TABLE Eleve ADD CONSTRAINT uq_eleve UNIQUE (nom, prenom, date_naiss);
ALTER TABLE Livre ADD CONSTRAINT chk_genre CHECK (genre IN ('Roman', 'Poésie', 'Théâtre'));
ALTER TABLE Livre DROP CONSTRAINT fk_livre_auteur;
```

```
SET FOREIGN_KEY_CHECKS = 0;
-- opérations temporaires
SET FOREIGN_KEY_CHECKS = 1;
```

DISABLE / ENABLE CONSTRAINT

SQL standard et Oracle. Avec XAMPP : FOREIGN_KEY_CHECKS.

Supprimer : les filles avant les mères

1 · Emprunt

2 · Livre

3 · Eleve

4 · Auteur

filles, puis mères

```
DROP TABLE Emprunt;
DROP TABLE Livre;
DROP TABLE Auteur;  -- accepté
DROP DATABASE nom_base;
```

DROP est définitif

Structure et données disparaissent.
Exporter avant.

Refusé tant que Livre existe :

DROP TABLE Auteur; → une table fille la référence encore

Exporter et importer au format SQL

Exporter

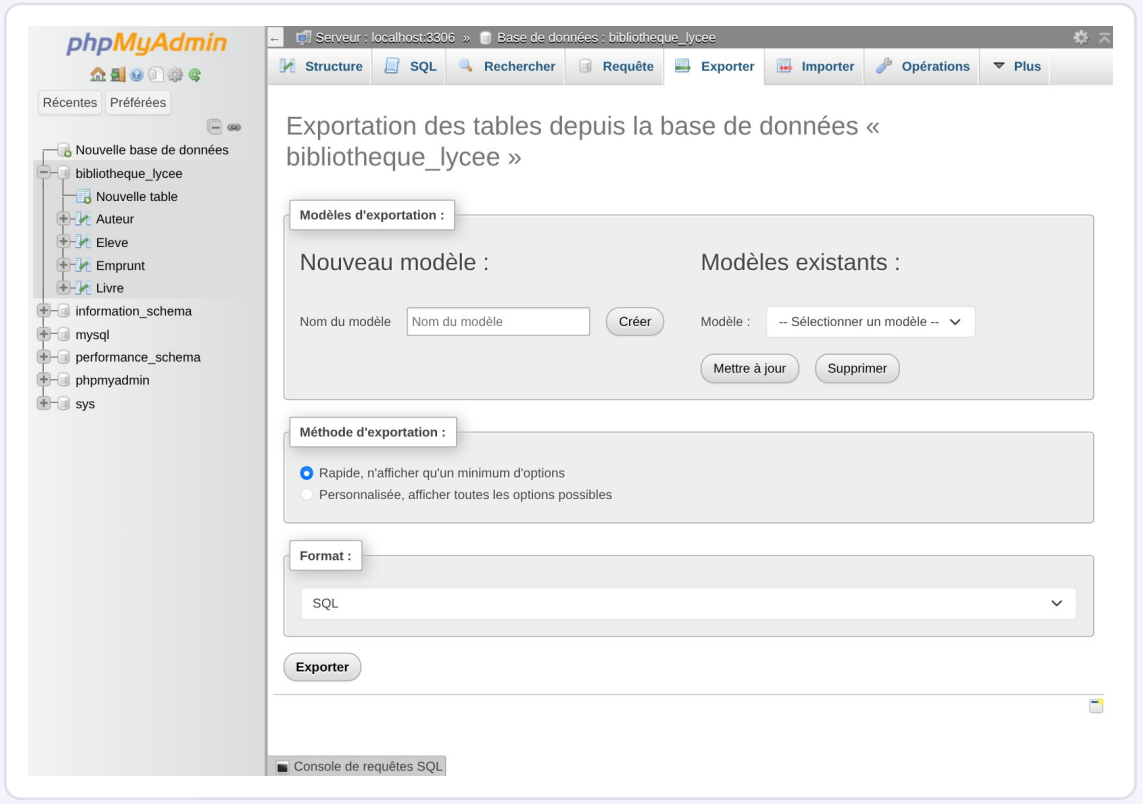
- Sélectionner la base, onglet Exporter
- Méthode Rapide, format SQL
- Ou Personnalisée : structure, données ou les deux
- Exporter : bibliotheque_lycee.sql

Importer

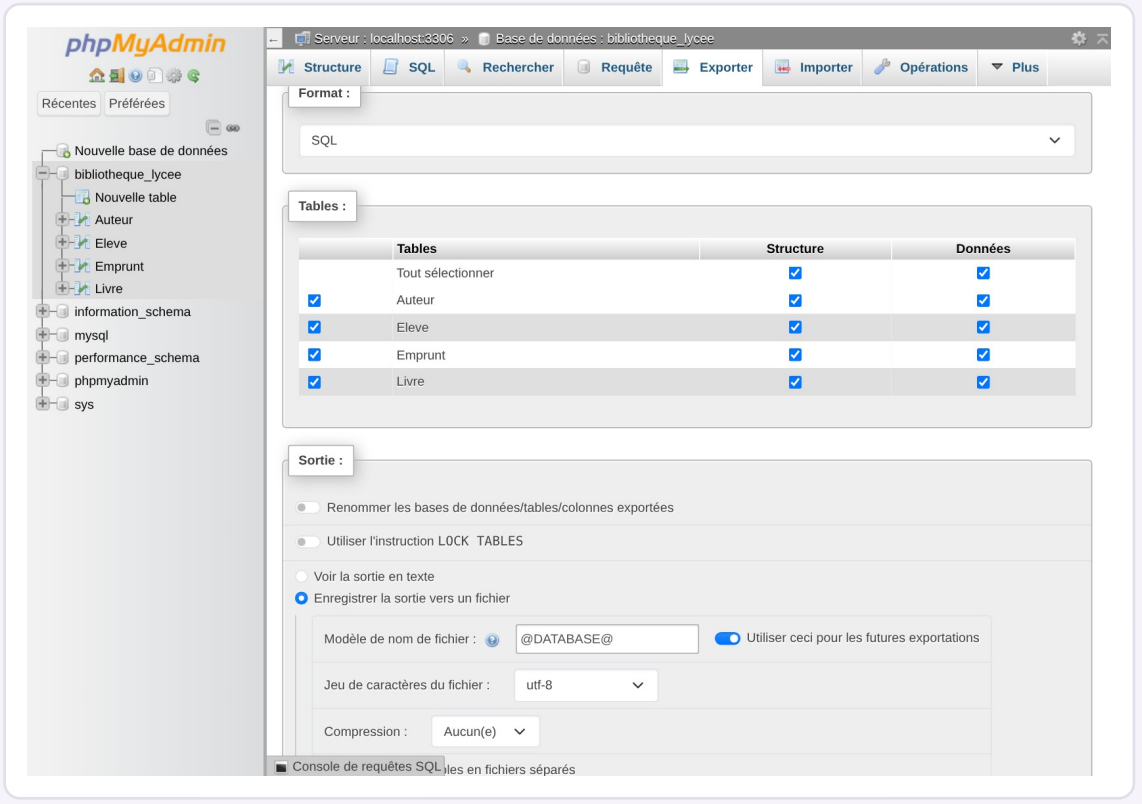
- Créer la base vide si le fichier n'a pas CREATE DATABASE
- Onglet Importer, choisir le fichier .sql
- Importer : phpMyAdmin exécute les requêtes
- Vérifier le nombre de lignes

```
-- Extrait du fichier
CREATE TABLE `Auteur` (
  `id_auteur` int(11)
    NOT NULL,
  ...
);
INSERT INTO `Auteur`
  VALUES (1,'Hugo',
  'Victor','France');
```

L'onglet Exporter, deux méthodes



Méthode rapide, format SQL



Méthode personnalisée : table par table

À VOUS DE JOUER

Exercices

Ex. 1

Faire évoluer Eleve

Ajouter telephone, agrandir nom à 80 caractères sans perdre NOT NULL.

Ex. 2

Contrainte de genre

Limiter genre à Roman, Poésie, Théâtre ; tester une insertion refusée.

Ex. 3

Sauvegarde

Exporter la base, supprimer les tables dans le bon ordre, réimporter.

À retenir

- ALTER : ADD, DROP, MODIFY, CHANGE, RENAME COLUMN
- Nommer les contraintes pour les supprimer
- Supprimer les filles avant les mères
- Exporter avant tout DROP

SÉANCE 9

Insérer, modifier, supprimer des données

Chapitre 4

Objectifs

- Ajouter des lignes avec INSERT
- Modifier et supprimer avec UPDATE et DELETE
- Lire un message d'erreur du SGBD

INSERT ajoute des lignes

```
INSERT INTO Eleve VALUES
  (107, 'Ayari', 'Nour', '4SI1', '2008-12-01');

INSERT INTO Eleve (num_eleve, nom, prenom, classe)
  VALUES (108, 'Sassi', 'Ines', '3SI1');

INSERT INTO Auteur (id_auteur, nom, prenom, nationalite) VALUES
  (8, 'Mahfouz', 'Naguib', 'Egypte'),
  (9, 'Darwich', 'Mahmoud', 'Palestine');
```

Règles d'écriture

- Textes et dates entre apostrophes
- Nombres sans apostrophes, point décimal
- Apostrophe doublée : 'L''Etranger'
- La ligne mère avant la ligne fille

Le SGBD refuse les données incohérentes

Code	Signification	Contrainte
1062	Valeur déjà utilisée	PRIMARY KEY, UNIQUE
1364	Valeur obligatoire manquante	NOT NULL
4025	Condition non respectée	CHECK
1452	Ligne mère inexistante	FOREIGN KEY
1451	Ligne mère encore référencée	FOREIGN KEY

```
INSERT INTO Emprunt VALUES
  (9, 99, 101, '2026-10-13', NULL);
```

ERROR 1452 (23000)

Cannot add or update a child row: a foreign key constraint fails (fk_emprunt_livre)

Le livre 99 n'existe pas.

UPDATE modifie des valeurs existantes

```sql
UPDATE Livre SET prix = prix * 1.10 WHERE genre = 'Roman';
```

Avant

titre	prix
Les Misérables	25.500
L'Etranger	9.800
La Peste	11.200
Le Livre des jours	13.500



Après

titre	prix
Les Misérables	**28.050**
L'Etranger	**10.780**
La Peste	**12.320**
Le Livre des jours	**14.850**

Sans WHERE, UPDATE modifie toutes les lignes de la table.

DELETE supprime des lignes

```sql
-- 1. Vérifier
SELECT id_emprunt, date_retour FROM Emprunt
WHERE date_retour < '2026-10-01';
-- 2. Supprimer
DELETE FROM Emprunt
WHERE date_retour < '2026-10-01';
```

id_emprunt	date_retour
1	2026-09-29
3	2026-09-27
5	2026-09-24

3 lignes supprimées

Commande	Ce qui disparaît	Ce qui reste
DELETE FROM t WHERE ...;	Les lignes qui vérifient la condition	Les autres lignes et la table
DELETE FROM t;	Toutes les lignes	La table vide
DROP TABLE t;	La table : structure et données	Rien

ON DELETE CASCADE : la suppression se propage

```
DELETE FROM Eleve WHERE num_eleve = 101;
```

Eleve

num_eleve	nom
~~101~~	~~Ben Salah~~
102	Trabelsi
103	Gharbi
104	Mansour
105	Jaziri
106	Haddad

ON DELETE CASCADE

Les emprunts de l'élève 101 sont supprimés avec lui : 8 → 6

Emprunt

id_emprunt	id_livre#	num_eleve#
~~1~~	~~2~~	~~101~~
~~2~~	~~3~~	~~101~~
3	5	102
4	7	103
5	4	104
6	6	105
7	1	103
8	8	102

À VOUS DE JOUER

Exercices

Ex. 1 **Nouveaux membres**
Insérer l'élève 107 complet et l'auteur 7 sans nationalité ; vérifier les valeurs par défaut.

Ex. 2 **Lire les erreurs**
Provoquer les erreurs 1062, 1364 et 1452 ; nommer la contrainte en cause.

Ex. 3 **Retour de livre**
Enregistrer le retour de l'emprunt 2 au 13 octobre ; vérifier avec SELECT.

Ex. 4 **Nettoyage**
Supprimer les emprunts rendus avant le 1er octobre, après un SELECT de contrôle.

À retenir

- INSERT : une ligne mère avant ses filles
- UPDATE et DELETE : toujours un WHERE
- SELECT de contrôle avant de modifier
- Code d'erreur = contrainte violée

SÉANCE 10

SELECT et les opérateurs

Chapitre 5, sections 5.1 et 5.2

Objectifs

- Écrire SELECT … FROM … WHERE … ORDER BY
- Utiliser les opérateurs de comparaison et logiques
- Éviter les pièges de NULL et de AND / OR

Ordre d'écriture, ordre d'exécution

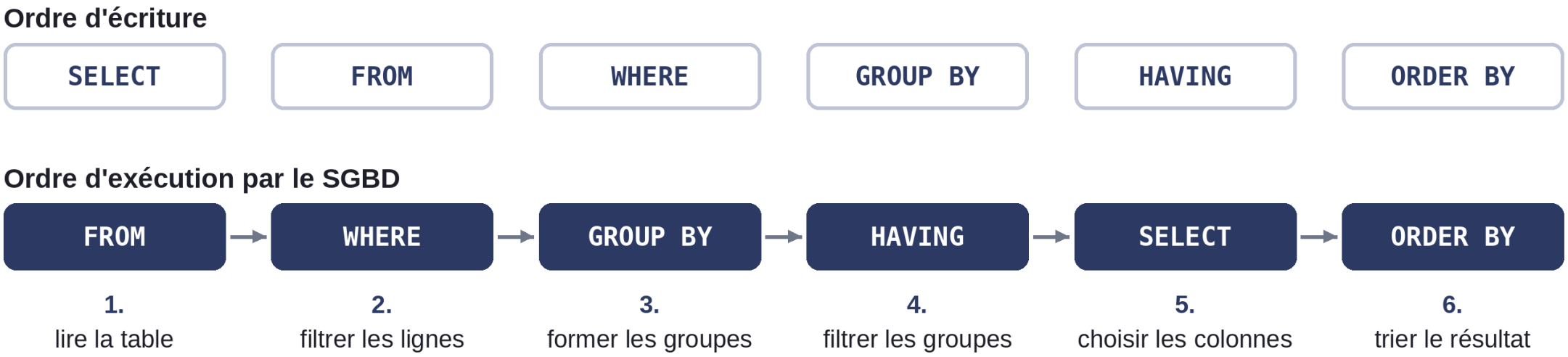
```
SELECT [DISTINCT] colonnes [AS alias]
FROM table
[WHERE condition]
[GROUP BY critère]
[HAVING condition]
[ORDER BY expression [ASC|DESC]];
```

Un alias de SELECT est inconnu dans WHERE.

Ordre d'exécution

1. **FROM**
2. **WHERE**
3. **GROUP BY**
4. **HAVING**
5. **SELECT**
6. **ORDER BY**

Le SGBD lit d'abord la table



FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Choisir les colonnes, renommer, trier

```
SELECT titre, annee AS parution
FROM Livre ORDER BY annee DESC;
```

titre	parution
Les Chants de la vie	1955
Le Barrage	1955
La Peste	1947
L'Etranger	1942
Le Livre des jours	1929
Le Prophète	1923
Les Misérables	1862
Les Contemplations	1856

```
SELECT DISTINCT genre FROM Livre;
```

→ Roman · Poésie · Théâtre (3 lignes au lieu de 8)

```
SELECT titre, prix * 0.8 AS prix_solde FROM Livre;
```

Filtrer les lignes avec WHERE

Opérateur	Exemple
= <> > < >= <=	annee >= 1950
AND, OR, NOT	genre = 'Roman' AND prix < 12
IN (liste)	nationalite IN ('Tunisie', 'Liban')
BETWEEN a AND b	annee BETWEEN 1900 AND 1950
LIKE motif	titre LIKE 'Le%'
IS [NOT] NULL	date_retour IS NULL

Motifs LIKE

Motif	Exemple trouvé
'Le%'	Le Barrage
'%s'	Les Misérables
'%de%'	Les Chants de la vie
'_a%'	Mansour, Jaziri, Haddad

% : 0 ou plusieurs caractères · _ : exactement un

Deux pièges : NULL et la priorité de AND

NULL

```
WHERE date_retour = NULL
-- 0 ligne, toujours
```

```
WHERE date_retour IS NULL
-- 3 livres non rendus
```

AND avant OR

```
WHERE genre = 'Roman' OR
  genre = 'Poésie' AND prix < 10
```

6 lignes

```
WHERE (genre = 'Roman' OR
  genre = 'Poésie') AND prix < 10
```

3 lignes

En cas de doute : des parenthèses.

À VOUS DE JOUER

Exercices

Ex. 1 **Catalogue trié**
Titres et prix des livres, du plus cher au moins cher.

Ex. 2 **Auteurs ciblés**
Auteurs tunisiens ou libanais, avec IN, puis avec OR.

Ex. 3 **Recherche**
Livres dont le titre contient « de », parus entre 1900 et 1960.

Ex. 4 **Non rendus**
Numéros des emprunts non rendus ; expliquer pourquoi = NULL échoue.

À retenir

- SELECT … FROM … WHERE … ORDER BY
- BETWEEN inclut les bornes ; % et _ dans LIKE
- IS NULL, jamais = NULL
- AND avant OR : parenthèses

SÉANCE 11

Dates, agrégats et regroupements

Chapitre 5, section 5.3 · Chapitre 6, section 6.1

Objectifs

- Utiliser les fonctions de date
- Calculer avec COUNT, SUM, AVG, MIN, MAX
- Grouper avec GROUP BY, filtrer avec HAVING

Calculer avec les dates

Fonction	Rôle
DAY MONTH YEAR	Partie d'une date
NOW()	Date et heure courantes
ADDDATE(d, n)	Ajoute n jours
DATEDIFF(d1, d2)	Jours de d2 à d1
DATE(ch)	Date d'un DATETIME

```
SELECT id_emprunt,
  DATEDIFF(date_retour, date_emprunt) AS nb_jours
FROM Emprunt WHERE date_retour IS NOT NULL;
```

id_emprunt	nb_jours
1	14
3	7
4	16
5	14
8	7

Résumer des lignes en une valeur

```
SELECT COUNT(*), MIN(prix), MAX(prix), AVG(prix), SUM(prix) FROM Livre;
```

COUNT(*)	MIN(prix)	MAX(prix)	AVG(prix)	SUM(prix)
8	7.900	25.500	12.3000000	98.400

Écriture	Ce qui est compté	Valeur
COUNT(*)	Toutes les lignes de Emprunt	**8**
COUNT(date_retour)	Les lignes où date_retour n'est pas NULL	**5**
COUNT(DISTINCT num_eleve)	Les élèves différents	**5**

GROUP BY : une ligne de résultat par groupe

```
SELECT genre, COUNT(*) AS nb_livres, AVG(prix) AS prix_moyen FROM Livre GROUP BY genre;
```

Roman — **4 livres** 25.500 · 9.800 · 11.200 · 13.500

Poésie — **3 livres** 12.000 · 8.500 · 7.900

Théâtre — **1 livre** 10.000



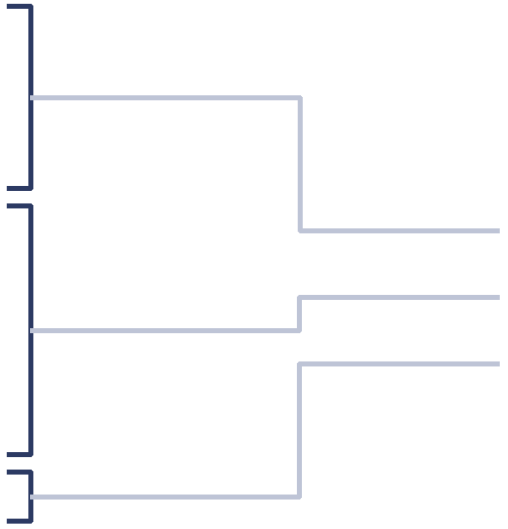
genre	nb_livres	prix_moyen
Roman	4	15.0000000
Poésie	3	9.4666667
Théâtre	1	10.0000000

Comment GROUP BY forme les groupes

```
SELECT genre, COUNT(*) AS nb_livres, AVG(prix) AS prix_moyen FROM Livre GROUP BY genre;
```

Livre

titre	genre	prix
Les Contemplations	Poésie	12.000
Les Chants de la vie	Poésie	8.500
Le Prophète	Poésie	7.900
Les Misérables	Roman	25.500
L'Etranger	Roman	9.800
La Peste	Roman	11.200
Le Livre des jours	Roman	13.500
Le Barrage	Théâtre	10.000



Résultat

genre	nb_livres	prix_moyen
Poésie	3	9.467
Roman	4	15.000
Théâtre	1	10.000

GROUP BY genre : un groupe par genre, une ligne de résultat par groupe

WHERE filtre les lignes, HAVING les groupes

```
SELECT genre, COUNT(*) AS nb FROM Livre WHERE annee > 1900 GROUP BY genre HAVING COUNT(*) >= 2;
```

WHERE

6 livres après 1900

GROUP BY

Poésie 2 · Roman 3 · Théâtre 1

HAVING

Groupes ≥ 2 : Théâtre éliminé

WHERE	HAVING
Filtre les lignes, avant le groupement	Filtre les groupes, après le groupement
Sans fonction d'agrégation	Avec, en général, une fonction d'agrégation

À VOUS DE JOUER

Exercices

Ex. 1 **Emprunts de septembre**
Emprunts de septembre 2026 avec MONTH et YEAR.

Ex. 2 **Retour prévu**
Date de retour prévue (15 jours) des emprunts non rendus avec ADDDATE.

Ex. 3 **Statistiques**
Nombre de livres, prix moyen, livre le moins cher du fonds.

Ex. 4 **Bons lecteurs**
Élèves ayant emprunté au moins deux livres (GROUP BY, HAVING).

À retenir

- DATEDIFF(récente, ancienne) > 0
- Agrégat sans GROUP BY : une seule ligne
- Colonne non agrégée → dans GROUP BY
- WHERE avant, HAVING après le groupement

SÉANCE 12

Jointures et requêtes imbriquées

Chapitre 6, sections 6.2 et 6.3

Objectifs

- Écrire une jointure entre deux ou trois tables
- Éviter le produit cartésien
- Utiliser une sous-requête avec IN, NOT IN ou une comparaison

La jointure relie clé étrangère et clé primaire

```
SELECT L.titre, A.nom
FROM Livre L, Auteur A
WHERE L.id_auteur = A.id_auteur
  AND A.nationalite = 'Tunisie';
```

```
FROM Livre L JOIN Auteur A ON L.id_auteur = A.id_auteur
```

Écriture équivalente avec JOIN … ON

titre	nom
Les Chants de la vie	Chebbi
Le Barrage	Messadi

n tables

→ n − 1 conditions de jointure

clé étrangère = clé primaire

Chaque livre retrouve la ligne de son auteur

```
SELECT L.titre, A.nom FROM Livre L, Auteur A WHERE L.id_auteur = A.id_auteur;
```

Livre

titre	id_auteur#
Les Misérables	1
Les Contemplations	1
L'Etranger	2
La Peste	2
Les Chants de la vie	3
Le Barrage	4
Le Prophète	5
Le Livre des jours	6

Auteur

id_auteur	nom
1	Hugo
2	Camus
3	Chebbi
4	Messadi
5	Gibran
6	Hussein

chaque livre est associé
à la ligne de son auteur

Résultat

titre	nom
Les Misérables	Hugo
Les Contemplations	Hugo
L'Etranger	Camus
La Peste	Camus
Les Chants de la vie	Chebbi
Le Barrage	Messadi
Le Prophète	Gibran
Le Livre des jours	Hussein

Sans condition de jointure : le produit cartésien

8 × 6 = 48

livres × auteurs = lignes sans signification

```
SELECT COUNT(*) FROM Livre, Auteur;
-- 48
```

```
... WHERE L.id_auteur = A.id_auteur
-- 8 : un auteur par livre
```

Compter les tables, compter les conditions.

Trois tables, deux conditions de jointure

```
SELECT E.nom, E.prenom, E.classe, L.titre, M.date_emprunt
FROM Emprunt M, Eleve E, Livre L
WHERE M.num_eleve = E.num_eleve AND M.id_livre = L.id_livre
  AND M.date_retour IS NULL ORDER BY M.date_emprunt;
```

nom	prenom	classe	titre	date_emprunt
Ben Salah	Leila	4SI1	L'Etranger	2026-10-01
Jaziri	Omar	4SI2	Le Barrage	2026-10-05
Gharbi	Amel	4SI2	Les Misérables	2026-10-06

Une sous-requête fournit une liste ou une valeur

Une liste : IN, NOT IN

```
SELECT nom, prenom FROM Eleve
WHERE num_eleve NOT IN
  (SELECT num_eleve FROM Emprunt);
```

nom	prenom
Haddad	Sami

Une valeur : =, >, <

```
SELECT titre, prix FROM Livre
WHERE prix >
  (SELECT AVG(prix) FROM Livre);
```

titre	prix
Les Misérables	25.500
Le Livre des jours	13.500

Avec =, >, < : une seule valeur. Sinon : IN.

La sous-requête s'exécute en premier

```
SELECT titre FROM Livre WHERE id_livre IN
   (SELECT id_livre FROM Emprunt WHERE date_retour IS NULL);
```

Étape 1 : Emprunt

id_emprunt	id_livre	date_retour
1	2	2026-09-29
2	**3**	**NULL**
3	5	2026-09-27
4	7	2026-10-08
5	4	2026-09-24
6	**6**	**NULL**
7	**1**	**NULL**
8	8	2026-10-09

la sous-requête s'exécute en premier

liste renvoyée

`(3, 6, 1)`

Étape 2 : Livre

id_livre	titre
1	**Les Misérables**
2	Les Contemplations
3	**L'Etranger**
4	La Peste
5	Les Chants de la vie
6	**Le Barrage**
7	Le Prophète
8	Le Livre des jours

résultat : les 3 livres actuellement empruntés

Lire de l'intérieur vers l'extérieur

```
SELECT nom, prenom FROM Eleve WHERE num_eleve IN
  (SELECT num_eleve FROM Emprunt WHERE id_livre IN
    (SELECT id_livre FROM Livre WHERE id_auteur IN
      (SELECT id_auteur FROM Auteur WHERE nationalite = 'Tunisie')));
```

Auteurs tunisiens

3, 4

→

Leurs livres

5, 6

→

Emprunteurs

102, 105

→

Résultat

Trabelsi Karim · Jaziri Omar

À VOUS DE JOUER

Exercices

Ex. 1 **Livres et auteurs**
Titre et nom de l'auteur de chaque livre, avec WHERE puis avec JOIN … ON.

Ex. 2 **Emprunts par classe**
Nombre d'emprunts par classe (jointure et GROUP BY).

Ex. 3 **Auteurs disponibles**
Auteurs dont aucun livre n'est actuellement emprunté (NOT IN).

Ex. 4 **Le plus ancien**
Titre du livre le plus ancien avec une sous-requête et MIN.

À retenir

- Jointure : clé étrangère = clé primaire
- n tables → n − 1 conditions
- Sous-requête : IN pour une liste, = > < pour une valeur
- Lire les sous-requêtes de l'intérieur

MÉMO SQL

L'essentiel en une page

Définir (LDD)

CREATE DATABASE / TABLE

ALTER TABLE … ADD, DROP, MODIFY

ADD / DROP CONSTRAINT

DROP TABLE / DATABASE

Manipuler (LMD)

INSERT INTO t (cols) VALUES (…)

UPDATE t SET col = … WHERE …

DELETE FROM t WHERE …

Interroger (LID)

SELECT … FROM … WHERE …

GROUP BY … HAVING …

ORDER BY … ASC | DESC

Jointure : FK = PK

Sous-requête : IN, NOT IN